Java로 API 활용하기 — 학습지

Nov 8, 2026 · @수현

들어가며

이 학습지를 끝까지 따라오면 **인터넷에서 포켓몬 정보를 받아와 능력치 순위를 매기고, 두 마리를 배틀시키는 프로그램**을 직접 만들게 됩니다. 포켓몬 데이터는 하나도 우리가 입력하지 않아요. 이미 누군가 만들어 둔 것을 빌려 씁니다. 그 방법이 바로 **API**예요.

교시	배울 것	만들 것
1교시	API란 무엇인가	주사위, 콘솔 경마
2교시	웹 API에 요청 보내기	피카츄 정보를 받아 화면에 찍기
3교시	JSON에서 값 꺼내기	피카츄의 키·몸무게·능력치 출력
4교시	API를 클래스로 나누기	포켓몬 도감, 포켓몬 카드 검색기
5교시	배운 것 합치기	포켓몬 배틀 도감

준비물 JDK 11 이상, 인터넷, 크롬 같은 브라우저

파일 관리 수업 동안 만드는 파일(`Api.java`, `JsonUtil.java`, `Pokemon.java`, `PokeApi.java` …)은 **모두 같은 폴더(프로젝트)에** 두세요. 앞에서 만든 클래스를 뒤에서 계속 씁니다.

빈칸 문제 푸는 법

각 교시 끝에 **빈칸 문제**가 있어요. 코드에서 `____①____` 처럼 비어 있는 곳에 들어갈 말을 찾아 쓰면 됩니다. 답은 바로 앞에서 배운 내용 안에 있어요. 막히면 앞을 다시 보세요.

난이도: ★ 앞에 나온 걸 그대로 옮기면 돼요 · ★★ 조금 응용 · ★★★ 생각이 필요해요

답을 채운 뒤에는 **꼭 직접 실행해 보세요.** 컴퓨터가 채점해 줍니다.

1교시 — API란 무엇인가

> **이번 시간 목표** 우리는 첫 수업부터 이미 API를 쓰고 있었다는 것을 알아차리기.
> `System.out.println`도, `Scanner`도 남이 만들어 둔 기능을 정해진 방법대로 빌려 쓴 것이에요.

생각해 보기

휴대폰 날씨 앱은 하늘에 온도계를 직접 달아 봤을까요?

아니에요. 기상 데이터를 가진 곳에 "대구 지금 몇 도예요?"라고 물어보고, 답을 받아서 보여줄 뿐이에요. 이렇게 **프로그램이 다른 프로그램에게 정해진 방법으로 부탁하는 창구**를 API라고 해요.

식당에 비유하면 이렇습니다.

식당	프로그래밍
손님	내 프로그램
메뉴판	API 설명서(문서)
주문	요청(메서드 호출, 웹 요청)
주방	남이 만든 코드나 서버 — 안을 몰라도 돼요
나온 음식	결과값(리턴값, 응답)

손님은 주방에서 불을 어떻게 다루는지 몰라도 메뉴판만 보고 주문하면 음식을 받아요. API도 같습니다. **안이 어떻게 생겼는지 몰라도 사용법만 알면 쓸 수 있어요.**

API를 왜 쓸까?

API는 Application Programming Interface의 줄임말이에요. "프로그램끼리 만나는 접점"이라는 뜻입니다.

1. **바퀴를 다시 만들지 않으려고** — 난수 만들기, 문자열 자르기를 매번 직접 짜면 시간이 끝없이 들어요.
2. **이미 검증된 코드라서** — 수많은 개발자가 써 보며 버그를 잡아 둔 코드예요.
3. **내가 가질 수 없는 데이터를 쓰려고** — 날씨, 환율, 포켓몬 정보는 내가 직접 모을 수 없어요. 2교시부터 다룰 웹 API가 이 경우예요.

자바 표준 API를 쓰는 두 가지 방법

방법	모양	예	이유
클래스 이름으로 바로 부르기	`클래스.메서드()`	`Math.max(3, 7)`	기억할 것이 없는 기능이라 객체가 필요 없어요 (static 메서드)
객체를 만들고 부르기	`new` 후 `객체.메서드()`	`random.nextInt(6)`	객체가 자기 상태를 갖고 일해요. 우리가 만든 클래스와 같은 방식

`Math`, `String`, `System`, `Thread`는 import 없이 쓸 수 있고, `Random`, `Scanner`는 `import java.util.Random;`처럼 위치를 알려줘야 해요.

메뉴판 보는 법 검색창에 "java Random api"를 치면 오라클 공식 문서가 나와요. `nextInt(int bound)` 설명에서 "0부터 bound 바로 앞까지"라는 말을 찾아 보세요. 주사위를 만들 때 왜 `+ 1`을 붙이는지 알 수 있어요.

따라 하기 1 — 표준 API 네 가지

```
import java.util.Random;

public class StdApiDemo {
    public static void main(String[] args) throws Exception {
        Random random = new Random();

        // 1) Random: 1~6 주사위
        int dice = random.nextInt(6) + 1;
        System.out.println("주사위: " + dice);

        // 2) Math: 큰 값, 절댓값, 반올림
        System.out.println("더 큰 값: " + Math.max(17, 42));
        System.out.println("절댓값: " + Math.abs(-7));
        System.out.println("반올림: " + Math.round(3.6));

        // 3) String: 글자 수, 대문자, 포함 여부
        String word = "java api";
        System.out.println("길이: " + word.length());
        System.out.println("대문자: " + word.toUpperCase());
        System.out.println("api 포함? " + word.contains("api"));

        // 4) Thread.sleep: 0.5초 쉬면서 카운트다운
        for (int i = 3; i >= 1; i--) {
            System.out.println(i + "...");
            Thread.sleep(500);
        }
        System.out.println("출발!");
    }
}
```

기억할 것

- `random.nextInt(6)`은 0~5를 줘요. 주사위를 만들려면 `+ 1`.
- `Thread.sleep(500)`의 단위는 밀리초(1000 = 1초)예요. 프로그램을 잠깐 멈춰서 게임에 움직임을 줄 때 써요.
- `main` 뒤의 `throws Exception`은 "이 코드는 문제가 생길 수도 있다"는 표시예요. `sleep`과 웹 요청에 꼭 필요해요. **빼면 `sleep`에 빨간 줄이 생겨요.**

따라 하기 2 — 콘솔 경마

`Random`으로 전진 칸 수를 정하고 `sleep`으로 화면을 천천히 바꾸면 경주가 움직이는 것처럼 보여요. `"-".repeat(n)`도 `String` API예요.

```
import java.util.Random;

public class HorseRace {
    public static void main(String[] args) throws Exception {
        String[] names = {"번개", "질풍", "천둥", "바람"};
        int[] pos = new int[names.length];
        int goal = 30;
        Random random = new Random();

        boolean finished = false;
        while (!finished) {
            for (int i = 0; i < names.length; i++) {
                pos[i] += random.nextInt(3) + 1;          // 1~3칸 전진
                if (pos[i] >= goal) { pos[i] = goal; finished = true; }
                System.out.println(names[i] + "\t|" + "-".repeat(pos[i]) +
"🐎");
            }
            System.out.println("================================");
            Thread.sleep(400);
        }
    }
}
```

도전 (빨리 끝났다면)

- [ ] 말을 `Horse` 클래스(이름, 위치)로 바꾸고 `Horse[]` 배열로 관리하기
- [ ] 경주가 끝나면 위치 기준으로 정렬해 1~4등 출력하기
- [ ] 시작 전에 Scanner로 우승 예상 말을 입력받고, 맞혔는지 알려주기

1교시 빈칸 문제

1-1 ★ 1부터 6까지 나오는 주사위를 만드세요.

```
Random random = new Random();
int dice = random.____①____(6) ____②____ 1;
```

1-2 ★ 두 점수 중 더 큰 값을 구하세요. 반복문 없이 표준 API 한 줄로.

```
int score1 = 72, score2 = 85;
int best = ____①____.____②____(score1, score2);
```

1-3 ★★ "3, 2, 1"을 **1초씩** 쉬면서 출력하세요. (main에 `throws Exception`을 잊지 마세요!)

```
for (int i = 3; i >= 1; i--) {
    System.out.println(i);
    Thread.____①____(____②____);
}
```

1-4 ★★ 빈칸에 `new로 객체를 만들어서` 또는 `클래스 이름으로 바로` 중 하나를 쓰세요.

코드	어떻게 부르나
`Math.abs(-7)`	①
`random.nextInt(10)`	②
`sc.nextInt()` (Scanner)	③
`Thread.sleep(500)`	④

1-5 ★★★ 아래 코드는 "0~9 중 하나"를 만들려는데 컴파일 오류가 나요. 왜일까요? 어떻게 고치면 될까요?

```
int n = Random.nextInt(10);
```

2교시 — 포켓몬 API에 첫 요청 보내기

> **이번 시간 목표** 웹 API는 "인터넷 너머에 있는 메뉴판"이에요. 주소를 보내면 답이 와요. 자바에서는 `HttpClient` 라는 전화기로 주문하고, 답은 숫자가 아니라 긴 글자 덩어리(JSON)로 와요.

생각해 보기

포켓몬 도감 앱을 만들려면 포켓몬 1,000마리 정보를 우리가 직접 다 입력해야 할까요?

아니에요. 이미 누군가 정리해 두고 "필요하면 주소로 물어봐"라고 열어 둔 곳이 있어요. 그게 **PokeAPI** 입니다. 가입도 비밀번호(키)도 필요 없는 무료 API예요.

	1교시 표준 API	오늘부터 웹 API
어디에 있나	내 컴퓨터의 자바 안	인터넷 너머 다른 컴퓨터(서버)
어떻게 부르나	`Math.max(3, 7)` 같은 메서드 호출	주소(URL)를 보내기
결과 모양	`int`, `String` 같은 자바 값	긴 글자 덩어리(JSON)
실패할 수 있나	거의 없음	인터넷이 끊기거나, 없는 주소면 실패

1단계 — 브라우저로 먼저 열어보기

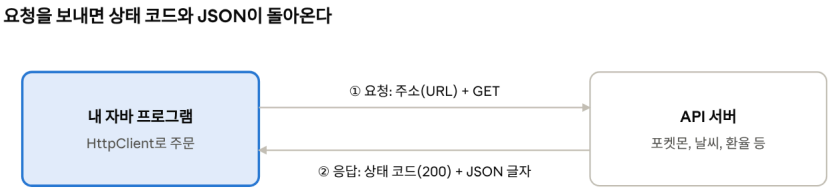
크롬 주소창에 아래 주소를 넣어 보세요.

```
https://pokeapi.co/api/v2/pokemon/pikachu
```

중괄호와 따옴표가 가득한 글자가 나와요. 이게 피카츄의 모든 정보예요. 다음을 해 보세요.

1. `Ctrl + F` 로 `"height"` 를 찾아보세요. 옆에 있는 숫자는? (피카츄 키)
2. `"weight"` 도 찾아보세요. (몸무게)
3. 주소 끝의 `pikachu` 를 `charmander` (파이리)로 바꿔 다시 열어보세요.
4. `pikachuu` 처럼 일부러 틀리게 써 보세요. 무엇이 나오나요?

여기서 알아야 할 것은 하나예요. **브라우저도 결국 "주소로 요청을 보내고, 돌아온 답을 화면에 보여주는 프로그램"**이라는 것. 오늘은 브라우저가 하는 일을 자바로 직접 해 봅니다.



요청과 응답 · 한 번의 왕복

웹 API를 부르는 것은 늘 이 한 번의 왕복이에요. 오늘 쓰는 모든 코드는 ①을 만들고 ②를 읽는 일입니다.

2단계 — 주소에도 규칙이 있다

부분	예	뜻	식당으로 치면
규칙	`https://`	안전한 방식으로 대화하겠다	전화로 주문
서버 주소	`pokeapi.co`	어느 컴퓨터에 물어볼지	어느 식당
버전	`/api/v2`	API 메뉴판의 2판	메뉴판 2판
종류	`/pokemon`	무엇에 대해 물을지	메뉴 종류(면류)
이름	`/pikachu`	그중 어느 것	짜장면

마지막 이름 부분만 바꾸면 다른 포켓몬이 나와요. 그래서 코드에서 `"https://pokeapi.co/api/v2/pokemon/" + name` 처럼 문자열을 붙여 주소를 만들 수 있어요.

주의 포켓몬 이름은 **영어 소문자**로 써야 해요. `피카츄`, `Pikachu`는 안 돼요. 도감 번호도 돼요 (`/pokemon/25` = 피카츄).

주소를 보내는 것을 **요청(request)**, 돌아오는 것을 **응답(response)**이라고 해요. 응답에는 내용과 함께 **상태 코드**라는 숫자가 붙어 와요. 주문이 잘 됐는지 알려주는 신호예요.

상태 코드	뜻	식당으로 치면
200	성공	음식 나왔습니다
404	없음	그런 메뉴는 없어요
429	너무 많은 요청	주문 좀 천천히 해 주세요
500	서버 오류	주방에 문제가 생겼어요

3단계 — 자바로 주문하기: 4단계

브라우저가 한 번에 하는 일을 자바에서는 4단계로 나눠서 해요.

단계	식당 비유	자바에서 쓰는 클래스
① 주소 준비	무엇을 시킬지 정하기	`String`
② 전화기 준비	전화기 장만	`HttpClient`
③ 주문서 작성	주문서에 주소와 방식 적기	`HttpRequest`
④ 주문하고 답 받기	전화 걸고 음식 받기	`HttpResponse`

① 주소 준비 — 그냥 문자열이에요.

```
String url = "https://pokeapi.co/api/v2/pokemon/pikachu";
```

② 전화기 준비 — 인터넷으로 말을 걸 도구를 만들어요. `new` 대신 `newHttpClient()`라는 메서드로 만들는 것만 다르고, 결국 객체를 하나 받는 거예요.

```
HttpClient client = HttpClient.newHttpClient();
```

③ 주문서 작성 — 어디로(`uri`), 어떻게(`GET` = "주세요") 보낼지 적어요.

```
HttpRequest request = HttpRequest.newBuilder()   // 빈 주문서 꺼내기
        .uri(URI.create(url))                    // 주소 칸 채우기
        .GET()                                   // 방식 칸 채우기
        .build();                                // 주문서 완성!
```

`.`으로 메서드를 줄줄이 잇는 모양이 낯설죠? "양식 채우기"라고 생각하세요. 빈 양식(`newBuilder`)을 꺼내서 칸을 하나씩 채우고(`uri`, `GET`) 마지막에 완성(`build`)해요. 줄을 바꾼 건 읽기 쉽게 하려는 것뿐이고, 한 줄로 써도 같아요. `URI.create(url)`는 문자열을 "자바가 인정하는 주소 객체"로 바꿔요.

④ 주문하고 답 받기 — 전화기(`client`)로 주문서(`request`)를 보내고, 답이 올 때까지 기다려요.

```
HttpResponse<String> response =
        client.send(request, HttpResponse.BodyHandlers.ofString());
```

`BodyHandlers.ofString()`은 "답을 `String`으로 포장해 주세요"라는 뜻이에요. 그래서 변수 타입도 `HttpResponse<String>`, 즉 "내용이 String인 응답"입니다.

다 합치면

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class PokeFirstCall {
    public static void main(String[] args) throws Exception {
        // ① 주소: 피카츄 정보를 주세요
        String url = "https://pokeapi.co/api/v2/pokemon/pikachu";
```

```
        // ② 전화기 준비
        HttpClient client = HttpClient.newHttpClient();

        // ③ 주문서 작성
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .GET()
                .build();

        // ④ 주문하고 답 받기
        HttpResponse<String> response =
                client.send(request, HttpResponse.BodyHandlers.ofString());

        // 결과 확인 (내용이 아주 길어서 앞 300글자만)
        System.out.println("상태 코드: " + response.statusCode());
        System.out.println(response.body().substring(0, 300));
    }
}
```

기억할 것

- `import` 네 줄: 이 클래스들은 `java.net` 안에 있어서 위치를 알려줘야 해요.
- `throws Exception` : 인터넷 요청은 실패할 수 있어서 꼭 붙여야 해요.
- `substring(0, 300)` : 피카츄 정보는 수만 글자라 통째로 찍으면 콘솔이 꽉 차요. 전체 길이가 궁금하면 `response.body().length()` 를 찍어 보세요.

실행하면 브라우저에서 본 것과 같은 글자가 콘솔에 찍혀요. 그런데 키랑 몸무게만 뽑고 싶다면? → 3교시에서 배워요.

4단계 — 도우미 클래스 Api 만들기

왜 만들까? 앞으로 파이리, 꼬부기, 잠만보도 불러야 해요. 매번 ②~④을 복사해 붙이면 길고, 하나를 고칠 때 전부 고쳐야 해요. 반복되는 코드는 메서드로 묶는다! 이미 배운 원칙이에요.

무엇을 받고 무엇을 돌려줄까? 주소(`String`)를 받아서 답 내용(`String`)을 돌려줘요. 그러면 메서드 모양은 `static String get(String url)` .

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

// 웹 API를 부르는 코드를 한곳에 모아 둔 도우미 클래스
public class Api {
    // 전화기는 하나만 만들어 두고 계속 쓴다
    private static final HttpClient client = HttpClient.newHttpClient();

    // url로 요청을 보내고, 답 내용(JSON 글자)을 돌려준다
    public static String get(String url) throws Exception {
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .GET()
                .build();
        HttpResponse<String> response =
                client.send(request, HttpResponse.BodyHandlers.ofString());
        if (response.statusCode() != 200) {
            System.out.println("요청 실패! 상태 코드: " +
response.statusCode());
        }
        return response.body();
    }
}
```

이제 어디서든 한 줄이면 돼요.

```
String json = Api.get("https://pokeapi.co/api/v2/pokemon/snorlax");
```

1교시의 `Math.max()` 처럼 클래스 이름으로 바로 부르는 `static` 메서드를 **우리가 직접 만든 거예요**. "API를 쓰는 사람"에서 "API를 만드는 사람"으로 한 발 옮긴 겁니다. `Api.java` 는 5교시까지 계속 써요.

2교시 빈칸 문제

2-1 ★ 이브이(eevee) 정보를 받는 주소를 완성하세요.

```
https://pokeapi.co/api/v2/____①____/____②____
```

2-2 ★★ 4단계 코드의 빈칸을 채우세요.

```
HttpClient client = HttpClient.____①____();

HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.____②____(url))
        .____③____()
        .____④____();

HttpResponse<String> response =
        client.____⑤____(request, HttpResponse.BodyHandlers.ofString());

System.out.println("상태 코드: " + response.____⑥____());
System.out.println("내용: " + response.____⑦____());
```

2-3 ★★ `https://pokeapi.co/api/v2/pokemon/pikachuu` 로 요청하면 상태 코드는 몇이 나올까요? 그 뜻은?

2-4 ★★ 도우미 클래스를 써서 도감 번호 133번 포켓몬의 정보를 받으세요.

```
String json = ____①____.____②____("https://pokeapi.co/api/v2/pokemon/" +
____③____);
```

3교시 — JSON에서 값 꺼내기

> **이번 시간 목표** JSON은 "이름표가 붙은 값들의 묶음"이에요. 이름표 위치를 찾고(`indexOf`), 그 뒤를 잘라내고(`substring`), 숫자로 바꾸면(`parseInt`) 원하는 값이 나와요. 세 개 모두 이미 아는 표준 API예요.

1단계 — JSON은 어떻게 생겼을까

피카츄 정보에서 우리가 쓸 부분만 추려 보기 좋게 줄을 바꿨어요. (실제 응답은 훨씬 길고, 줄바꿈·띄어쓰기 없이 한 줄로 와요.)

```
{
  "height": 4,
  "id": 25,
  "name": "pikachu",
  "stats": [
    { "base_stat": 35, "stat": { "name": "hp" } },
    { "base_stat": 55, "stat": { "name": "attack" } },
    { "base_stat": 40, "stat": { "name": "defense" } },
    { "base_stat": 50, "stat": { "name": "special-attack" } },
    { "base_stat": 50, "stat": { "name": "special-defense" } },
    { "base_stat": 90, "stat": { "name": "speed" } }
  ],
  "weight": 60
}
```

기호	뜻	자바로 치면
`{ }`	객체 — 이름표 붙은 값들의 묶음	객체 하나
`"이름": 값`	이름표와 값 한 쌍	필드와 그 값
`[ ]`	여러 개를 순서대로	배열

기호	뜻	자바로 치면
`"글자"`	문자열	`String`
`35` , `true` , `null`	숫자, 참거짓, 없음	`int` , `boolean` , `null`

나란히 놓고 보면 똑같아요.

```
JSON  →  {"name": "pikachu", "height": 4}
자바  →  Pokemon p = new Pokemon("pikachu", 4);
```

JSON은 객체를 글자로 적어서 인터넷으로 보내기 위한 형식이에요. 인터넷으로는 객체를 그대로 보낼 수 없어서 글자로 바꿔 보내고, 받는 쪽은 다시 객체로 바꿔서 써요. 오늘은 **값 하나를 꺼내는 것**까지, 다음 시간에는 **객체로 다시 만드는 것**까지 해요.

2단계 — 짧은 글자로 손 연습

피카츄 응답은 수만 글자라 눈으로 셈 수 없어요. 그래서 **20글자짜리 가짜 JSON**으로 먼저 연습해요. 원리는 똑같아요.

```
{"hp":35,"speed":90}
```

자바 문자열로 쓰기 — 문자열 안에 `"` 를 넣으려면 앞에 `\` 를 붙여야 해요. `\"` 는 화면에 `"` 한 글자로 보여요.

```
String s = "{\"hp\":35,\"speed\":90}";
```

칸 번호 매기기 — 글자마다 0부터 번호를 붙여요. 이 표가 오늘의 핵심이에요.

칸	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
글자	{	"	h	p	"	:	3	5	,	"	s	p	e	e	d	"	:

이제 스피드 값 `90` 을 다섯 단계로 꺼내요. 한 줄씩 표를 보며 숫자를 확인하세요.

```
public class HandParse {
    public static void main(String[] args) {
        String s = "{\"hp\":35,\"speed\":90}";

        int pos = s.indexOf("\"speed\":");          // ① 이름표 위치
        int start = pos + "\"speed\":".length();    // ② 값이 시작하는 위치
        int end = s.indexOf("}", start);            // ③ 값이 끝나는 위치
        String value = s.substring(start, end);     // ④ 잘라내기 → "90"
        int speed = Integer.parseInt(value);        // ⑤ 숫자로 바꾸기

        System.out.println("pos=" + pos + ", start=" + start + ", end=" +
end);
        System.out.println("스피드: " + speed);
    }
}
```

실행 결과: `pos=9, start=17, end=19` , `스피드: 90`

단계	하는 일	왜 그 숫자가 나올까
① `indexOf`	`"speed":` 가 시작하는 칸을 찾아요	9번 칸의 `"` 부터 시작
② `+ length()`	이름표 길이만큼 건너뛰어요	`"speed":` 는 8글자 → 9 + 8 = 17
③ `indexOf("}", start)`	17번부터 `}` 를 찾아요	19번 칸
④ `substring(17, 19)`	17번부터 19번 **앞까지** 자라요	17, 18번 칸 = "90"
⑤ `parseInt`	글자 "90"을 숫자 90으로	글자는 더하기·비교를 못 하니까

꼭 기억할 것 두 가지

- `substring(시작, 끝)` 의 끝은 **포함되지 않아요**. 제일 많이 틀리는 부분이에요.
- 왜 `parseInt` 가 필요할까요? `"90" + 10` 을 찍어 보세요. `100` 이 아니라 `9010` 이 나와요.

바로 해 보기 같은 방법으로 `hp` 값 35를 꺼내 보세요. `}` 를 찾으면 어떻게 될까요? hp 뒤에는 무엇이 있나요? → 여기서 **값의 끝은 `,` 또는 `}` 또는 `]` 중 먼저 나오는 것**이라는 규칙을 찾을 수 있어요.

3단계 — 도우미 메서드 JsonUtil.getValue

왜 만들까? 키, 몸무게, HP, 공격력… 꺼낼 때마다 다섯 줄을 매번 쓰면 길어요. 2교시에 `Api.get()` 을 만든 것과 같은 이유예요.

무엇을 받고 무엇을 돌려줄까? JSON 글자와 이름표를 받아서, 값을 `String` 으로 돌려줘요. 못 찾으면 `null`.

```
// JSON 글자에서 원하는 값만 꺼내는 아주 단순한 도우미
// (진짜 JSON 해석기는 아님 — 수업용)
public class JsonUtil {

    // json 안에서 "key": 다음 값을 처음부터 찾아 꺼낸다
    public static String getValue(String json, String key) {
        return getValue(json, key, 0);
    }

    // fromIndex 위치부터 찾는다 (같은 key가 여러 번 나올 때 사용)
    public static String getValue(String json, String key, int fromIndex) {
        String pattern = "\"" + key + "\":";
        int start = json.indexOf(pattern, fromIndex);
        if (start == -1) return null;              // 못 찾음
        start = start + pattern.length();

        while (json.charAt(start) == ' ') start++; // 공백 건너뛰기

        if (json.charAt(start) == '"') {           // 문자열 값: "..." 사이
            int end = json.indexOf('"', start + 1);
            return json.substring(start + 1, end);
        }
        int end = start;                            // 숫자·true/false 값: , }
] 앞까지
        while (end < json.length() && ",}]".indexOf(json.charAt(end)) == -1)
end++;
        return json.substring(start, end).trim();
    }
}
```

새로운 코드가 아니라, 2단계의 다섯 줄을 어떤 이름표에나 쓸 수 있게 바꾼 거예요.

2단계 손 연습	getValue 안에서	달라진 점
`"\"speed\":"`	`"\"" + key + "\":"`	이름표를 매개변수로 받아 만들어요
`pos + 8`	`start + pattern.length()`	이름표 길이가 매번 다르니 `length()`
`indexOf("}")`	`,` `}` `]` 중 하나가 나올 때까지 한 칸씩	hp에서 찾은 규칙
없음	`"` 로 시작하면 따옴표 사이만	문자열 값(`"pikachu"`)도 꺼낼 수 있게
없음	`if (start == -1) return null;`	이름표가 없을 때 프로그램이 멈추지 않게

기억할 것

- `",}]".indexOf(글자) == -1` 은 "이 글자가 `,` `}` `]` 중 어느 것도 아니면"이라는 뜻이에요.
- 같은 이름의 메서드가 두 개예요(오버로딩). 매개변수 개수가 다르면 자바가 알아서 골라 불러요. 두 번째 것이 왜 필요한지는 5단계에서 나와요.
- `JsonUtil.java` 도 `Api.java` 와 같은 폴더에 두세요.

4단계 — 피카츄에 적용하기

```
String json = Api.get("https://pokeapi.co/api/v2/pokemon/pikachu");

int id = Integer.parseInt(JsonUtil.getValue(json, "id"));
int height = Integer.parseInt(JsonUtil.getValue(json, "height"));
int weight = Integer.parseInt(JsonUtil.getValue(json, "weight"));

System.out.println("도감 번호: " + id);
System.out.println("키: " + (height / 10.0) + "m");
System.out.println("몸무게: " + (weight / 10.0) + "kg");
```

왜 10으로 나눌까요? 피카츄 키가 `4`로 와요. 4m일 리는 없죠. PokeAPI 설명서를 보면 키는 10cm 단위, 몸무게는 100g 단위예요. 그래서 키 0.4m, 몸무게 6.0kg이 돼요. **API는 숫자만 주고, 단위는 설명서에 있어요.** 메뉴판을 읽어야 하는 이유죠.

`10`이 아니라 `10.0`으로 나누는 이유는? 정수끼리 나누면 `4 / 10 = 0`이 돼요.

5단계 — 같은 이름표가 여러 번 나올 때

능력치 `base_stat`은 HP, 공격, 방어, 특수공격, 특수방어, 스피드 순서로 **6번** 나와요. `getValue(json, "base_stat")`을 여섯 번 부르면 어떻게 될까요? 직접 해 보세요. 여섯 번 모두 35(HP)가 나와요. `indexOf`는 늘 **처음 만나는 것**을 찾기 때문이에요.

해결 방법: 찾을 때마다 **찾기 시작 위치를 뒤로 옮겨요.** 책에서 단어를 찾고, 다음 것을 찾을 때는 그 다음부터 읽는 것과 같아요.

```
String[] statNames = {"HP", "공격", "방어", "특수공격", "특수방어", "스피드"};
int from = 0;
for (int i = 0; i < 6; i++) {
    from = json.indexOf("\"base_stat\":", from);   // from부터 다음 base_stat 찾기
    String value = JsonUtil.getValue(json, "base_stat", from);
    System.out.println(statNames[i] + ": " + value);
    from++;   // 한 칸 옮겨야 같은 곳을 또 찾지 않는다
}
```

`from++`을 지우면? 직접 지워서 실행해 보세요. 무슨 일이 일어나나요?

3교시 빈칸 문제

3-1 ★ 아래 문자열에서 각 값을 구하세요. (2단계 표를 보고 세어도 돼요)

```
String s = "{\"hp\":35,\"speed\":90}";
```

식	값
`s.indexOf("\"hp\":")`	①
`"\"hp\":".length()`	②
`s.substring(6, 8)`	③
`s.indexOf(",")`	④

3-2 ★★ hp 값 35를 꺼내는 코드를 완성하세요. hp 뒤에는 무엇이 있는지 생각해 보세요.

```
int pos = s.____①____("\"hp\":");
int start = pos + "\"hp\":".____②____();
int end = s.indexOf("____③____", start);
int hp = Integer.____④____(s.substring(start, ____⑤____));
```

3-3 ★★ 잠만보(snorlax)의 몸무게를 kg으로 출력하세요.

```
String json = Api.get("https://pokeapi.co/api/v2/pokemon/____①____");
int weight = Integer.parseInt(JsonUtil.____②____(json, "____③____"));
System.out.println("몸무게: " + (weight / ____④____) + "kg");
```

3-4 ★★★ 피카츄의 **스피드**(여섯 번째 base_stat)만 출력하세요.

```
int from = 0;
int speed = 0;
for (int i = 0; i < ____①____; i++) {
    from = json.indexOf("\"base_stat\":", ____②____);
    speed = Integer.parseInt(JsonUtil.getValue(json, "base_stat", from));
    ____③____;
}
System.out.println("스피드: " + speed);
```

3-5 ★★★ 다음 코드는 10을 출력할까요? 실행하기 전에 결과를 예상하고, 10이 나오게 고쳐 보세요.

```
String h = "4";
System.out.println(h + 6);
```

4교시 — API를 클래스로 나누기

> **이번 시간 목표** API를 다루는 코드를 역할에 따라 클래스로 나눠요. 가져오는 담당(`Api`), 번역 담당(`PokeApi`), 데이터 담당(`Pokemon`). 그러면 `main` 은 JSON을 전혀 몰라도 `PokeApi.getPokemon("pikachu")` 한 줄로 포켓몬 객체를 받을 수 있어요.

1단계 — 다 main에 쓰면 안 될까?

3교시까지 배운 대로 포켓몬 3마리 도감을 만들면 main이 이렇게 돼요.

```
// 나쁜 예: 모든 걸 main에
String json1 = Api.get("https://pokeapi.co/api/v2/pokemon/pikachu");
int hp1 = ... (indexOf, base_stat 반복 6줄)
int speed1 = ...

String json2 = Api.get("https://pokeapi.co/api/v2/pokemon/charmander");
int hp2 = ... (똑같은 6줄 또)
int speed2 = ...

String json3 = ... (똑같은 코드 또)
```

생각해 보세요.

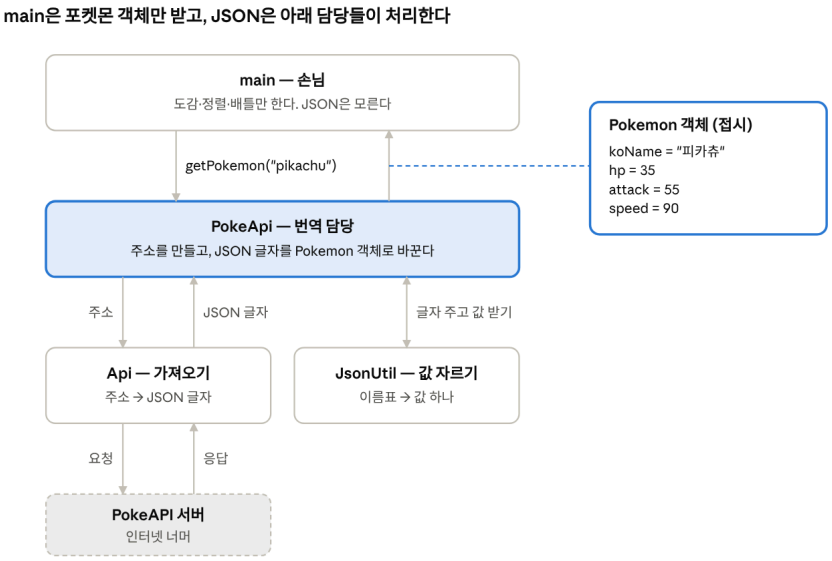
1. 포켓몬이 100마리면?
2. 능력치 순서를 잘못 알고 있었다면 몇 군데를 고쳐야 할까요?
3. 조원 중 JSON 처리를 모르는 친구도 배틀 기능을 만들 수 있을까요?
4. `hp1` , `hp2` , `hp3` … 포켓몬 한 마리 정보가 변수 여러 개로 흩어져 있으면, 정렬할 때 이름과 능력치를 함께 옮길 수 있을까요?

4번은 클래스를 처음 배울 때 이미 나온 이야기예요. `name1, score1, name2, score2` 를 `Student` 객체로 묶었던 것과 같아요. 오늘은 그 생각을 **데이터뿐 아니라 "하는 일"에도** 적용해요.

2단계 — 세 가지 역할

배달 앱으로 해외 음식을 시켰다고 생각해 봐요.

- **배달 기사** — 가게에 가서 봉투를 받아 와요. 안에 뭐가 든지는 관심 없어요. → `Api`
- **현지 메뉴를 아는 직원** — 외국어 메뉴판을 읽고, 봉투를 열어 우리가 알아보는 접시에 옮겨 담아요. → `PokeApi`
- **접시에 담긴 음식** — 우리가 실제로 먹는 것. → `Pokemon`
- **손님** — 접시만 보고 먹어요. 봉투가 어떻게 포장됐는지 몰라도 돼요. → `main`



포켓몬 도감의 클래스 구조 · 요청은 내려가고 객체가 올라온다

왼쪽 화살표는 "부탁"이 내려가는 길, 오른쪽 화살표는 "결과"가 올라오는 길이에요. JSON 글자는 PokeApi에서 멈추고, main에는 객체만 도착해요.

클래스	하는 일	알고 있는 것	몰라도 되는 것
`Api` (2교시)	주소를 받아 JSON 글자를 가져와요	HttpClient 사용법	포켓몬이 뭔지, JSON 안에 뭐가 있는지
`JsonUtil` (3교시)	JSON 글자에서 값 하나를 자라요	`indexOf`, `substring`	어떤 API에서 온 글자인지
`PokeApi` (오늘)	주소를 만들고, JSON을 `Pokemon` 객체로 바꿔요	PokeAPI 주소 규칙, 이름표, 단위	HttpClient를 어떻게 쓰는지
`Pokemon` (오늘)	포켓몬 한 마리 정보를 담아요	이름, HP, 공격력 …	인터넷, JSON, API가 있다는 것 자체
`main`	도감·정렬·배틀 같은 프로그램 기능	`PokeApi.getPokemon()` 한 줄	JSON, 주소, 이름표 전부

이렇게 나누면 좋은 점 세 가지

1. **고칠 곳이 하나예요.** 능력치 꺼내는 방법이 틀렸으면 `PokeApi` 한 곳만 고치면 포켓몬 100마리가 다 맞게 돼요.
2. **나눠서 만들 수 있어요.** 조원 A는 `PokeApi`, 조원 B는 배틀, 조원 C는 정렬. `Pokemon` 클래스 모양만 먼저 정해 두면 동시에 작업할 수 있어요.
3. **다른 API에도 같은 틀을 써요.** 포켓몬 카드 API를 쓰고 싶으면 `Api`와 `JsonUtil`은 그대로 두고, 번역 담당과 데이터 담당만 새로 만들면 돼요. (6단계에서 해 봐요.)

3단계 — 데이터 담당 Pokemon 클래스

접시부터 정해요. "무엇을 담을지"가 정해져야 "JSON에서 무엇을 꺼낼지"가 정해져요. 피카츄 JSON에는 수백 가지 정보가 있지만, 우리 도감과 배틀에 필요한 것만 골라요.

```
// 포켓몬 한 마리의 정보를 담는 클래스 (데이터 담당)
public class Pokemon {
    String name;      // 영어 이름 (API 주소에 쓰는 이름)
    String koName;    // 한국어 이름
    int id;           // 도감 번호
    int hp;
    int attack;
    int defense;
    int speed;
    double height;    // 키 (m)
    double weight;    // 몸무게 (kg)
```

```
    Pokemon(String name, String koName, int id, int hp, int attack,
            int defense, int speed, double height, double weight) {
        this.name = name;
        this.koName = koName;
        this.id = id;
        this.hp = hp;
        this.attack = attack;
        this.defense = defense;
        this.speed = speed;
        this.height = height;
        this.weight = weight;
    }

    // 정렬 기준을 글자로 받아서 그 능력치를 돌려준다
    int getStat(String kind) {
        switch (kind) {
            case "hp":      return hp;
            case "attack":  return attack;
            case "defense": return defense;
            default:        return speed;
        }
    }

    void print() {
        System.out.println("No." + id + " " + koName + "(" + name + ")"
                + " | HP " + hp + " 공격 " + attack + " 방어 " + defense
                + " 스피드 " + speed + " | " + height + "m " + weight + "kg");
    }
}
```

찾아보세요 이 클래스에 `Api`, `json`, `indexOf`가 나오나요? 하나도 없어요. 인터넷을 전혀 모르는 클래스예요.

- 키·몸무게는 이미 m·kg로 바뀐 값을 담아요. 단위 변환은 그 API를 아는 번역 담당의 일이니까요.
- `getStat("speed")`는 글자를 넣으면 그 능력치를 돌려줘요. 5교시에 정렬 기준을 고를 때 써요.

4단계 — 번역 담당 PokeApi 클래스

무엇을 받고 무엇을 돌려줄까? 영어 이름(`String`)을 받아 `Pokemon` 객체를 돌려줘요. 메서드 모양은 `static Pokemon getPokemon(String name)`.

안의 코드는 **새로 배우는 것이 거의 없어요**. 3교시에 main에 썼던 코드를 메서드 안으로 이사시키고, 마지막에 `System.out.println` 대신 `return new Pokemon(...)`을 하는 것만 달라요.

```
// PokeAPI와 대화하는 클래스 (번역 담당)
// JSON 글자를 받아서 Pokemon 객체로 바꿔 준다
public class PokeApi {
    static final String BASE = "https://pokeapi.co/api/v2/";

    // 영어 이름으로 포켓몬 한 마리를 불러온다
    static Pokemon getPokemon(String name) throws Exception {
        String json = Api.get(BASE + "pokemon/" + name);

        int id = Integer.parseInt(JsonUtil.getValue(json, "id"));
        double height = Integer.parseInt(JsonUtil.getValue(json, "height")) /
10.0; // 10cm 단위 → m
        double weight = Integer.parseInt(JsonUtil.getValue(json, "weight")) /
10.0; // 100g 단위 → kg

        // base_stat은 6번 나온다: HP, 공격, 방어, 특수공격, 특수방어, 스피드 순서
        int[] stats = new int[6];
        int from = 0;
        for (int i = 0; i < 6; i++) {
            from = json.indexOf("\"base_stat\":", from);
            stats[i] = Integer.parseInt(JsonUtil.getValue(json, "base_stat",
from));
            from++;
        }
```

```
        String koName = getKoreanName(name);
        return new Pokemon(name, koName, id, stats[0], stats[1], stats[2],
stats[5], height, weight);
    }

    // 한국어 이름은 pokemon-species 주소에 따로 있다
    static String getKoreanName(String name) throws Exception {
        String json = Api.get(BASE + "pokemon-species/" + name);

        int namesStart = json.indexOf("\"names\":[");
        if (namesStart == -1) return name;          // 못 찾으면 영어 이름이라도
        int ko = json.indexOf("\"name\":\"ko\"", namesStart);
        if (ko == -1) return name;

        int koNameAt = json.lastIndexOf("\"name\":", ko - 1);
        return JsonUtil.getValue(json, "name", koNameAt);
    }
}
```

코드	왜 이렇게 했을까
`static final String BASE`	주소 앞부분을 한 곳에 둡니다. 주소가 바뀌어도 여기만 고치면 돼요. `final` 은 "바뀌지 않는 값" 표시
`Api.get(BASE + "pokemon/" + name)`	2교시 주소 규칙 + 배달 기사에게 맡기기
`/ 10.0`	단위 변환은 이 API를 아는 번역 담당이 해요
`int[] stats = new int[6]`	여섯 개를 다 받아 놓고, 필요한 것(0·1·2·5번)만 골라 써요
`return new Pokemon(...)`	번역의 끝: 글자를 객체로. 이 순간부터 JSON은 필요 없어요
`return name;` (못 찾았을 때)	한국어 이름이 없어도 프로그램이 멈추지 않게

한국어 이름 찾기(`getKoreanName`) — 한국어 이름은 다른 주소(`pokemon-species`)에 여러 나라 이름과 함께 이렇게 들어 있어요.

```
"names":[ ... {"name":"피카츄","language":{"name":"ko", ...}}, ... ]
```

한국어 이름이 `"ko"` 표시보다 **앞에** 있어요. 그래서 `"ko"` 를 찾은 뒤 **뒤로 거슬러 올라가며** 찾는 `lastIndexOf` 를 써요. 지금은 "모양이 다르면 찾는 방법도 바꿔야 한다"는 것만 알면 돼요.

왜 `static` 일까? `PokeApi` 는 기억해 둘 것이 없어요. "피카츄 주세요"라고 하면 매번 새로 물어보면 돼요. 그래서 `Math.max()` 처럼 클래스 이름으로 바로 불러요.

5단계 — main에서 쓰기: 미니 도감

```
public class Pokedex {
    public static void main(String[] args) throws Exception {
        String[] names = {"pikachu", "charmander", "bulbasaur", "squirtle",
"eevee", "snorlax"};
        Pokemon[] list = new Pokemon[names.length];

        for (int i = 0; i < names.length; i++) {
            System.out.println(names[i] + " 불러오는 중...");
            list[i] = PokeApi.getPokemon(names[i]);
        }

        // 스피드가 빠른 순서로 버블 정렬
        for (int i = 0; i < list.length - 1; i++) {
            for (int j = 0; j < list.length - 1 - i; j++) {
                if (list[j].speed < list[j + 1].speed) {
                    Pokemon tmp = list[j];
                    list[j] = list[j + 1];
                    list[j + 1] = tmp;
                }
            }
```

```
        }

        System.out.println("\n=== 스피드 순위 ===");
        for (int i = 0; i < list.length; i++) {
            System.out.print((i + 1) + "위 ");
            list[i].print();
        }
    }
}
```

찾아보세요 이 main에 `json`, `indexOf`, `"base_stat"`, `https://` 같은 단어가 있나요? 없어요. main은 포켓몬 객체만 다뤄요. 정렬은 지난번에 배운 버블 정렬 그대로이고, 이름과 능력치가 객체 하나로 묶여 있어서 한 번에 같이 옮겨져요.

"불러오는 중..."이 천천히 찍히죠? 포켓몬 한 마리에 요청을 두 번(기본 정보 + 한국어 이름) 보내서 6마리면 12번이에요. 인터넷 요청에는 시간이 걸려요.

6단계 — 같은 틀로 포켓몬 카드 찾기

TCGdex는 포켓몬 카드를 찾아 주는 무료 API예요. 키가 필요 없고 한국어 카드도 있어요. 브라우저로 먼저 열어 보세요.

```
https://api.tcgdex.net/v2/ko/cards?name=피카츄
```

카드마다 `id`, `localId`, `name`, `image`가 있어요. `image` 주소 뒤에 `/high.png`를 붙여 브라우저에 넣으면 **실제 카드 그림**이 나와요! 결과가 비어 있으면 `ko`를 `en`으로, `피카츄`를 `pikachu`로 바꿔 보세요.

클래스 구조는 포켓몬 도감과 똑같아요.

역할	포켓몬 도감	포켓몬 카드
가져오는 담당	`Api`	`Api` (그대로)
값 자르기	`JsonUtil`	`JsonUtil` (그대로)
번역 담당	`PokeApi`	`TcgApi` (새로)
데이터 담당	`Pokemon`	`PokeCard` (새로)

```
// 포켓몬 카드 한 장 (데이터 담당)
public class PokeCard {
    String id;      // 카드 고유 번호 (예: swsh3-136)
    String name;    // 카드 이름
    String image;   // 카드 그림 주소 (없을 수도 있음)

    PokeCard(String id, String name, String image) {
        this.id = id;
        this.name = name;
        this.image = image;
    }

    void print() {
        System.out.println("[" + id + "] " + name);
        if (image != null) System.out.println("   그림: " + image +
"/high.png");
    }
}
```

```
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;

// TCGdex 포켓몬 카드 API와 대화하는 클래스 (번역 담당)
public class TcgApi {
    static final String BASE = "https://api.tcgdex.net/v2/ko/";   // ko = 한국
어 카드

    // 이름이 들어간 카드를 최대 max장까지 찾아 온다
```

```
    static PokeCard[] search(String name, int max) throws Exception {
        // 한글은 주소에 그대로 못 넣으므로 %ED%94... 같은 모양으로 바꾼다
        String q = URLEncoder.encode(name, StandardCharsets.UTF_8);
        String json = Api.get(BASE + "cards?name=" + q);

        PokeCard[] temp = new PokeCard[max];
        int count = 0;
        int from = 0;
        while (count < max) {
            from = json.indexOf("\"id\":", from);
            if (from == -1) break;                        // 더 이상 카드가 없음
            int next = json.indexOf("\"id\":", from + 1); // 다음 카드의 시작 위
치

            String id = JsonUtil.getValue(json, "id", from);
            String cardName = JsonUtil.getValue(json, "name", from);

            // 그림이 없는 카드도 있다: 다음 카드보다 뒤에서 찾은 image는 남의 것
            String image = null;
            int imageAt = json.indexOf("\"image\":", from);
            if (imageAt != -1 && (next == -1 || imageAt < next)) {
                image = JsonUtil.getValue(json, "image", from);
            }

            temp[count] = new PokeCard(id, cardName, image);
            count++;
            from++;
        }
        return Arrays.copyOf(temp, count);   // 찾은 개수만큼만 잘라서 돌려준다
    }
}
```

```
import java.util.Scanner;

public class CardSearch {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("찾을 포켓몬 이름 (예: 피카츄): ");
        String name = sc.next();

        PokeCard[] cards = TcgApi.search(name, 5);
        System.out.println(cards.length + "장 찾았어요!");
        for (PokeCard c : cards) c.print();
    }
}
```

새로 나온 것 세 가지

- **`URLEncoder.encode`** — 주소에는 영어·숫자·일부 기호만 들어갈 수 있어요. `피카츄`는 `%ED%94%BC%EC%B9%B4%EC%B8%84`로 바꿔서 보내요. 브라우저는 이 일을 몰래 해 주고 있었어요.
- **`image`가 없는 카드** — 그림이 없는 카드는 `image` 이름표가 없을 수 있어요. 그러면 `indexOf`는 다음 카드의 그림을 가져와 버려요. 그래서 "다음 카드의 `id`보다 앞에 있을 때만 내 것"인지 확인해요.
- **`Arrays.copyOf(temp, count)`** — 5칸짜리 배열을 만들었는데 3장만 찾았다면 뒤 2칸은 `null`이에요. 찾은 개수만큼 잘라 새 배열로 돌려주는 표준 API예요.

4교시 빈칸 문제

4-1 ★ 각 일을 누가 맡을까요? `Api` / `PokeApi` / `Pokemon` / `main` 중에서 고르세요.

하는 일	담당
`HttpClient`로 요청을 보낸다	①
키 `4`를 `0.4`m로 바꾼다	②
포켓몬의 공격력을 기억하고 있다	③
포켓몬 6마리를 스피드 순으로 정렬한다	④

하는 일	담당
`"base_stat"` 이름표를 알고 있다	⑤

4-2 ★ `Pokemon` 생성자의 빈칸을 채우세요.

```
Pokemon(String name, String koName, int id, int hp, int attack,
        int defense, int speed, double height, double weight) {
    ____①____.name = name;
    this.koName = ____②____;
    this.____③____ = hp;
    // ... 나머지도 같은 모양
}
```

4-3 ★★ `PokeApi.getPokemon`의 빈칸을 채우세요.

```
static ____①____ getPokemon(String name) throws Exception {
    String json = ____②____.get(BASE + "pokemon/" + ____③____);
    int id = Integer.parseInt(JsonUtil.getValue(json, "id"));
    // ... (능력치·키·몸무게 꺼내기)
    ____④____ new Pokemon(name, koName, id, stats[0], stats[1], stats[2],
stats[5], height, weight);
}
```

4-4 ★★ main에서 잠만보를 불러와 정보를 출력하고, HP가 100보다 크면 "튼튼하다!"를 출력하세요.

```
Pokemon p = ____①____.____②____("snorlax");
p.____③____();
if (p.____④____ > 100) System.out.println("튼튼하다!");
```

4-5 ★★ 포켓몬 카드에서 "리자몽"을 최대 3장 찾아 출력하세요.

```
PokeCard[] cards = ____①____.search("리자몽", ____②____);
for (PokeCard c : ____③____) c.print();
```

4-6 ★★★ 아래 두 클래스 중 하나는 `static` 메서드로, 하나는 `new`로 객체를 만들어 쓰는 게 맞아요. 어느 쪽이 어느 쪽이고, 왜 그럴까요?

- `PokeApi` — 이름을 주면 포켓몬 정보를 준다.
- `DeckApi` — 카드 덱을 섞고, 그 덱에서 한 장씩 뽑는다. 서버는 덱마다 `deck_id`를 주고, 다음에 뽑을 때 그 번호를 같이 보내야 한다.

5교시 — 프로젝트: 포켓몬 배틀 도감

> **이번 시간 목표** 4교시에 만든 `Pokemon` · `PokeApi`는 그대로 두고, 그 위에 **정렬(기준 고르기)과 1:1 배틀**을 올려요. API는 이미 클래스 뒤로 숨었으니, 오늘은 지금까지 배운 배열·정렬·반복문만으로 끝낼 수 있어요.

완성하면 이렇게 돼요 (숫자는 예시예요. 데미지에 난수가 들어가서 매번 달라요.)

```
정렬 기준 (hp / attack / defense / speed): speed
1위 No.25 피카츄(pikachu) | HP 35 공격 55 방어 40 스피드 90 | 0.4m 6.0kg
2위 No.4 파이리(charmander) | ...
...
싸울 포켓몬 두 마리 번호 (예: 1 3): 1 6
피카츄 vs 잠만보 배틀 시작!

피카츄의 공격! 29 데미지 → 잠만보 남은 HP 131
잠만보의 공격! 96 데미지 → 피카츄 남은 HP 0
피카츄 쓰러졌다! 잠만보 승리!
```

파일 구성과 조별 역할

새로 만드는 파일은 `Battle.java` 하나뿐이에요. 나머지는 지금까지 만든 걸 그대로 써요.

파일	만든 때	오늘 손대나요?
`Api.java`	2교시	그대로
`JsonUtil.java`	3교시	그대로
`Pokemon.java`	4교시	그대로 (`getStat` 사용)
`PokeApi.java`	4교시	그대로
`Battle.java`	오늘	새로 — 빈칸 뼈대에서 시작

`Battle.java`는 메서드 세 개로 나뉘어 있어서 조원 세 명이 나눠 맡을 수 있어요.

담당	메서드	하는 일	필요한 지식
조원 A	`sortBy`	고른 능력치가 높은 순으로 정렬	선택 정렬, `getStat`
조원 B	`fight`	두 포켓몬의 턴제 배틀	배열, 반복문, `Random`, `sleep`
조원 C	`main`	불러오기 → 기준 입력 → 정렬 → 배틀 연결	`PokeApi`, `Scanner`

세 명이 동시에 일할 수 있는 건 `Pokemon` 클래스 모양이 이미 정해져 있기 때문이에요. 4교시에 배운 "나눠서 만들 수 있다"를 직접 경험해 봐요.

배틀 규칙

1. 스피드가 빠른 포켓몬이 먼저 공격해요. 같으면 첫 번째로 고른 쪽.
2. 데미지 = 내 공격 − 상대 방어 ÷ 2 + (0~9 난수). 단, 최소 5.
3. 상대 HP가 0이 되면 끝. 아니면 차례를 바꿔요.

빈칸 뼈대 코드

빈칸을 다 채우면 돌아가는 프로그램이 돼요. 파일 맨 위에는 `import java.util.Random;`과 `import java.util.Scanner;`를 넣으세요.

5-1 ★★ 조원 A — 선택 정렬 (높은 순)

```
// kind 능력치가 높은 순으로 선택 정렬
static void sortBy(Pokemon[] list, String kind) {
    for (int i = 0; i < list.length - 1; i++) {
        int best = i;
        for (int j = ____①____; j < list.length; j++) {
            if (list[j].____②____(kind) > list[best].getStat(kind)) best =
j;
        }
        Pokemon tmp = list[i];
        list[i] = list[____③____];
        list[best] = ____④____;
    }
}
```

왜 `getStat(kind)`를 쓸까요? 지난번 정렬은 `list[j].speed`처럼 비교할 필드가 코드에 고정돼 있었어요. 그러면 공격력 정렬, 방어 정렬을 따로 만들어야 하죠. `getStat("attack")`처럼 기준을 글자로 넘기면 **정렬 코드 하나로 모든 기준**을 처리할 수 있어요.

5-2 ★★★ 조원 B — 턴제 배틀

```
static Random random = new Random();

// 두 포켓몬의 배틀. 이긴 포켓몬을 돌려준다
static Pokemon fight(Pokemon a, Pokemon b) throws Exception {
    // 스피드가 빠른 쪽이 먼저 공격
    Pokemon first = a, second = b;
    if (b.____①____ > a.speed) { first = b; second = a; }
    Pokemon[] p = {first, second};
    int[] hp = {p[0].hp, p[1].hp};   // 원래 HP를 지키려고 복사해서 쓴다
    int turn = 0;                    // 0번이 공격할 차례
```

```
    while (true) {
        int atk = turn, def = ____②____;
        int damage = p[atk].attack - p[def].defense / 2 + random.nextInt(10);
        damage = Math.____③____(damage, 5);            // 최소 5
        hp[def] = Math.max(hp[def] - damage, 0);

        System.out.println(p[atk].koName + "의 공격! " + damage + " 데미지 → "
                + p[def].koName + " 남은 HP " + hp[def]);
        Thread.sleep(600);

        if (hp[def] == ____④____) {
            System.out.println(p[def].koName + " 쓰러졌다! " + p[atk].koName +
" 승리!");
            return p[atk];
        }
        turn = ____⑤____;   // 차례 바꾸기
    }
}
```

이렇게 만든 이유

- **`p` 배열과 `turn`** — 공격하는 쪽과 맞는 쪽이 매 턴 바뀌어요. 두 마리를 배열에 넣고 번호(0과 1)만 바꾸면 같은 코드로 양쪽 공격을 다 처리해요.
- **`hp` 복사본** — `p[def].hp`를 직접 깎으면 배틀이 끝난 뒤 도감의 HP까지 0이 돼요. 객체는 도감 원본이니 건드리지 않아요.
- **`Math.max(값, 0)`** — HP가 음수로 찍히는 걸 막아요. 1교시의 `Math`가 여기서 쓰여요.

5-3 ★★ 조원 C — main 연결

```
public static void main(String[] args) throws Exception {
    String[] names = {"pikachu", "charmander", "bulbasaur", "squirtle",
"eevee", "snorlax"};
    Pokemon[] list = new Pokemon[names.length];
    for (int i = 0; i < names.length; i++) {
        list[i] = ____①____.getPokemon(names[i]);
    }

    Scanner sc = new Scanner(System.in);
    System.out.print("정렬 기준 (hp / attack / defense / speed): ");
    String kind = sc.next();
    ____②____(list, kind);
    for (int i = 0; i < list.length; i++) {
        System.out.print((i + 1) + "위 ");
        list[i].print();
    }

    System.out.print("\n싸울 포켓몬 두 마리 번호 (예: 1 3): ");
    Pokemon a = list[sc.nextInt() ____③____];
    Pokemon b = list[sc.nextInt() - 1];
    System.out.println(a.koName + " vs " + b.koName + " 배틀 시작!\n");
    fight(a, b);
}
```

확장 과제 (빨리 끝난 조)

- [ ] 내가 고른 포켓몬 넣기: 시작할 때 영어 이름 6개를 Scanner로 입력받아 `names`를 채우기 (★)
- [ ] 밸런스 조정: 잠만보가 한 방에 이기는 게 재미없다면 배틀 HP를 `hp * 3`으로 하거나 데미지 공식 바꾸기 (★)
- [ ] 토너먼트: 6마리를 2마리씩 싸우고 이긴 쪽끼리 다시 싸우기. `fight`가 승자를 `return`하는 이유가 여기서 나와요 (★★)
- [ ] 배틀 리그: 모든 포켓몬이 서로 한 번씩 싸우고, 승수를 배열에 세어 승수 순으로 정렬하기 (★★★)
- [ ] 카드도 보여주기: 우승한 포켓몬의 한국어 이름으로 `TcgApi.search(우승자.koName, 1)`을 불러 카드 그림 주소 출력하기 (★★)

발표 전 점검

- [ ] 기준을 바꾸면 순위가 바뀌나요?

- [ ] 배틀이 끝까지 진행되고 승자가 나오나요?
- [ ] 조원 각자 자기가 맡은 메서드를 한 문장으로 설명할 수 있나요?
- [ ] "main에 JSON이나 주소가 없는 이유"를 설명할 수 있나요?

도움말

오류가 날 때

이런 메시지가 나오면	원인	이렇게 해 보세요
`sleep` 이나 `send` 에 빨간 줄	`throws Exception` 이 빠짐	그 코드가 들어간 메서드 선언 끝에 `throws Exception` 붙이기
`package java.net.http does not exist`	자바 버전이 낮음	`java -version` 으로 확인. 11 이상이어야 해요
`UnknownHostException` , `ConnectException`	인터넷 끊김, 주소 오타	같은 주소를 브라우저로 먼저 열어 보기
"요청 실패! 상태 코드: 404"	포켓몬 이름 오타, 대문자, 한글 이름	영어 소문자로 (`pikachu`)
`NumberFormatException`	숫자가 아닌 글자를 숫자로 바꾸려 함	이름표 철자 확인. 같은 이름표가 앞에 또 있으면 `from` 으로 찾는 위치 옮기기
`NullPointerException`	`getValue` 가 이름표를 못 찾아 `null`	이름표 철자와 따옴표 확인
`StringIndexOutOfBoundsException`	응답이 예상과 다른 모양	`System.out.println(json)` 으로 받은 내용 먼저 찍어 보기
한글이 `???` 로 나옴	인코딩 설정	선생님께 여쭤보세요 (파일·콘솔을 UTF-8로)
포켓몬 카드 0장	한국어 카드 데이터가 없는 이름	`TcgApi` 의 `ko` 를 `en` 으로, 이름을 영어로

디버깅 습관 하나만 이상하면 일단 `json` 을 통째로 출력해 보세요. 받은 걸 눈으로 보면 원인이 대부분 보여요.

예의 지키기 PokeAPI는 모두에게 무료로 열려 있는 서비스예요. 반복문으로 같은 포켓몬을 수백 번 부르지 말고, 한 번 받은 객체는 배열에 담아 두고 쓰세요.

더 해 볼 재미있는 API

모두 키 없이 쓸 수 있고, 4교시의 틀(번역 담당 + 데이터 담당)을 그대로 적용할 수 있어요. 브라우저로 주소부터 열어 보고, 어떤 이름표가 있는지 확인한 뒤 만들어 보세요.

API	주는 것	주소 예	만들어 볼 것	주의
Deck of Cards	섞인 카드 덱, 뽑기	`https://deckofcardsapi.com/api/deck/new/shuffle/?deck_count=1`	하이로우, 블랙잭	서버가 덱을 기억해요 (`deck_id`)
Open Trivia DB	영어 퀴즈 (OX·객관식)	`https://opentdb.com/api.php?amount=5&type=boolean&encode=base64`	OX 퀴즈 대결	같은 곳에서는 5초에 한 번만. 글자는 자바 `Base64` 로 풀어요
Open-Meteo	날씨	`https://api.open-meteo.com/v1/forecast?latitude=35.87&longitude=128.60&current=temperature_2m`	도시별 기온 순위	같은 이름표가 두 번 나와요. `"current":{` 뒤부터 찾기

API	주는 것	주소 예	만들어 볼 것	주의
ExchangeRate-API	오늘의 환율	`https://open.er-api.com/v6/latest/USD`	환율 계산기, 여행 경비	화면에 출처(exchangerate-api.com) 표기